

Programação Orientada a Objetos

Trabalho Prático

Grupo 10

SpotiUM



Filipa Cosquete Santos
a106837



Cátia Alexandra Ribeiro da Eira
a107382



Andreia Alves Cardoso
a106915

Introdução

Este programa simula uma plataforma de gestão e reprodução de conteúdos musicais. A aplicação permite criar, organizar e reproduzir músicas, álbuns e listas de reprodução (playlists), bem como gerir os utilizadores que acedem à plataforma.

Os utilizadores podem ser ocasionais ou premium, com diferentes funcionalidades consoante o plano de subscrição associado. Utilizadores premium têm acesso à criação de playlists e liberdade na reprodução das músicas, e o programa pode gerar de forma automática listas de reprodução personalizadas, baseadas no seu histórico de reprodução, analisando os géneros de música e artistas que o utilizador prefere. Utilizadores ocasionais têm acesso limitado, com reprodução aleatória de conteúdos.

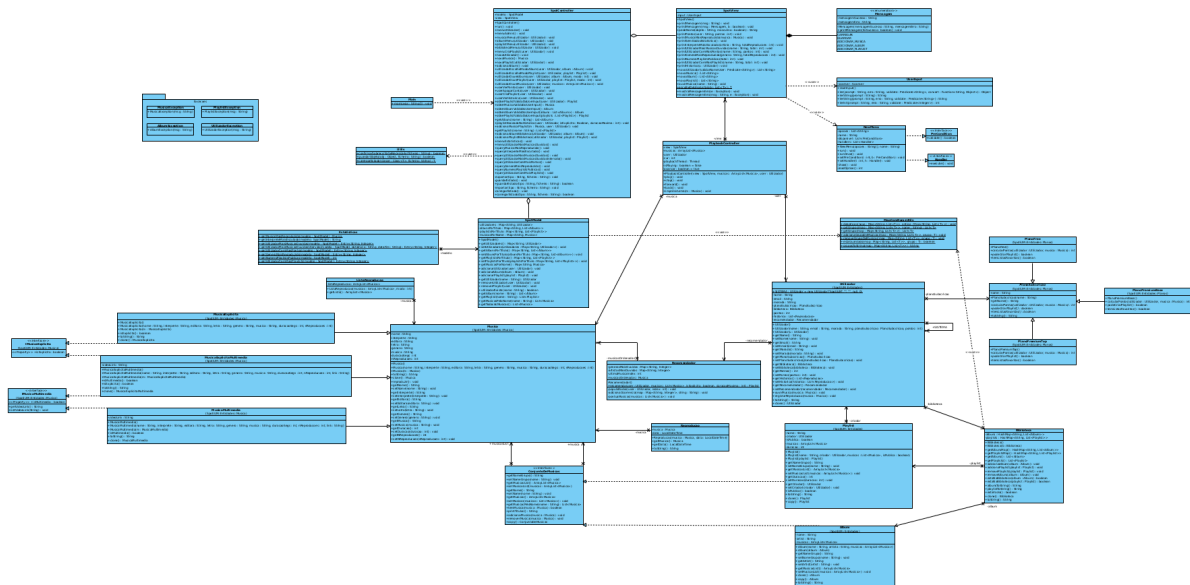
A aplicação suporta:

- Criação e gestão de utilizadores, músicas, álbuns e playlists;
- Associação de utilizadores a diferentes planos de subscrição;
- Reprodução de conteúdos com visualização da letra das músicas, de acordo com a duração da música;
- Registo de histórico de reprodução;
- Geração automática de playlists baseadas no histórico;
- Gravação e carregamento do estado da aplicação através de ficheiros;
- Queries.

Toda a interação com o utilizador é feita através de um sistema de menus em modo texto, código dado pelos professores. O programa tenta seguir os conceitos de programação orientada a objetos e usa várias funcionalidades de java. Foi construído com extensão futura em mente, logo apresenta modularidade e encapsulamento.

Arquitetura

Para este projeto escolhemos seguir uma estrutura MVC (Model - View - Controller) no âmbito de ter um programa modular, de modo a facilitar testagem e expansões futuras.



Para uma visualização com maior definição, consultar o ficheiro *SpotiUM.pdf*.

Entidades

→ Utilizador

Um utilizador possui informações básicas de identificação, como o nome, email e morada. O seu plano de subscrição irá afetar funções do programa, por exemplo, utilizadores com plano grátis apenas podem ouvir músicas em modo aleatório e não têm permissão para criar playlists. Além disso, o plano também influencia a quantidade de pontos que o utilizador recebe por interação com o sistema.

Cada utilizador tem uma biblioteca pessoal, onde pode guardar álbuns e playlists para acesso rápido. É possível adicionar playlists públicas de outros utilizadores à sua biblioteca, e todas as playlists que o utilizador cria são automaticamente guardadas na mesma.

O histórico do utilizador é uma lista de Reproduções, cada uma representando uma música juntamente com a data e hora em que foi ouvida.

O recomendador guarda um Map com counters de quantas vezes o utilizador ouviu cada género e artista. Este apenas é gerado quando o utilizador pede para gerar uma playlist, e o seu histórico é analisado. Guarda também um índice do número de músicas já analisadas, para evitar processar o histórico todas as vezes. Depois ele percorre todas as músicas do sistema e atribui-lhes pontos baseado no seu género e artista e então organiza-as numa lista ordenada de forma decrescente de pontos.

→ Música

A classe música representa a música base, contendo todos os campos requeridos pelo enunciado. A partir dela, foram criadas subclasses como Música Explícita, Música Multimédia e Música Explícita Multimédia, que estendem o seu comportamento. De modo a tornar a adição de novos tipos de música mais fácil e modular, foram definidas as interfaces *IMusicaMultimedia* e *IMusicaExplicita*, que contêm os métodos associados a cada tipo.

Dessa forma, uma música que combine múltiplas características pode simplesmente implementar as interfaces relevantes, sem a necessidade de ter que escolher que classes deve herdar ou duplicar código.

→ Álbum

Álbuns guardam as músicas presentes no sistema. Cada álbum tem um nome, artista e uma lista de músicas, que guarda por agregação.

→ Playlist

As playlists são coleções personalizadas de músicas, criadas por utilizadores ou geradas automaticamente de acordo com o histórico dos utilizadores. Cada playlist tem um nome, criador (utilizador ou o sistema) e ser pública ou privada. Utilizadores com plano grátis não podem criar uma playlist. Uma playlist pública pode ser adicionada à biblioteca de outros utilizadores.

Foi notado que ambas as playlists e os álbuns partilhavam o comportamento de guardar músicas, então a interface “**ConjuntoDeMusicas**” foi criada. Esta trata de todo o comportamento relacionado com inserir, procurar e apagar músicas de uma lista por agregação.

→ Lista de Reprodução

As listas de reprodução são simplesmente listas das músicas para tocar. Durante a sua criação é fornecida a lista em questão, e o modo de reprodução pretendido (a ordem já dada ou ordem aleatória). A lista resultante é então o que vai de facto ser tocado.

Exceções

O programa define várias exceções personalizadas, utilizadas principalmente para sinalizar erros relacionados com entidades inexistentes (e.g., quando se tenta aceder a um utilizador ou música que não existe). Isto permite separar a lógica de erro da lógica principal e apresentar mensagens mais informativas ao utilizador, garantindo também que o programa não falhe a meio.

MVC

O modelo irá guardar as instâncias de utilizadores, álbuns e playlists presentes no programa por composição. O controlador servirá como ponte entre o modelo e o *view*, sendo este a principal classe que gere o programa, uma vez que gere o flow do programa.

→ Modelo

O modelo é responsável por armazenar e gerir os dados centrais do programa, nomeadamente os utilizadores, álbuns e playlists. Os utilizadores são guardados num `Map<String, Utilizador>`, obrigando a unicidade do nome do utilizador. Por outro lado, os álbuns e playlists permitem nomes repetidos sendo armazenados em `Map<String, List<ConjuntoDeMusicas>>`.

Como esta lógica de armazenamento e manipulação de conjuntos de músicas se repete na gestão da biblioteca do utilizador, foi extraída para uma classe utilitária denominada “**MapComNomeUtils**”, que contém alguns métodos estáticos básicos como adição, pesquisa e remoção de conjuntos de músicas, entre outros.

→ View

A view trata da interação com o utilizador, sendo responsável tanto por recolher o input como por apresentar o output. Toda a interface é feita em modo texto, com menus simples e mensagens informativas. A view delega ao **UserInput** a validação de dados introduzidos, garantindo uma certa robustez nos inputs do utilizador.

→ Controlador

O controlador faz a mediação entre o modelo e a view. É aqui que reside a lógica de navegação da aplicação, composta essencialmente por menus. O controlador interpreta as opções do utilizador, coordenando a resposta do sistema — seja ao invocar métodos do modelo, seja ao atualizar a view.

◆ Controlador de Playback

Este controlador auxiliar trata da execução propriamente dita das músicas. Optámos por uma execução mais dinâmica de forma a ser mais relevante a duração das músicas. Assim, a letra de cada música vai sendo impressa linha a linha durante a respetiva duração.

Demonstração das Funcionalidades

A aplicação permite a interação com o utilizador através de menus baseados em texto, guiando-o pelas várias funcionalidades do sistema. Abaixo são apresentados exemplos ilustrativos de execução real do programa:

O menu principal:

```
*** Menu Inicial ***  
1 - Criar Utilizador  
2 - Login Utilizador  
3 - Funções de Administrador  
0 - Sair  
Opção:
```

Exemplo de criação de um Utilizador:

```
Opção: 1  
Nome: Jorge  
Morada: Rua 1  
Email: jg@gmail.com  
Plano:  
- 1. Plano grátis  
- 2. Plano Premium  
- 3. Plano TOP Premium  
3
```

O menu com funções administrativas:

```
*** Admin ***  
1 - Novo album  
2 - Importar dados  
3 - Exportar dados  
4 - Consultar Estatísticas  
0 - Sair
```

Exemplo de criação de um álbum

```
Opção: 1
Nome do Álbum: Deslocado
Nome do artista: NAPA
Número de músicas: 1
Nome da Música: Deslocado
Nome do artista: NAPA
Nome da editora: Universal Music Portugal
Letra: Conto os dias para mim\nCom a mala arrumada\n...
Género: Indie Pop
Ficheiro de música: deslocado.mp3
Duração (segundos): 180
É explícita?
1. Sim
2. Não 2
É multimédia?
1. Sim
2. Não 1
Link multimédia: deslocado.mp4
Álbum adicionado com sucesso!
```

Página de login:

```
Opção: 2
Nome do utilizador: Jorge

*** Utilizador Jorge ***
1 - Pesquisar música
2 - Pesquisar álbum
3 - Pesquisar playlist
4 - ---
5 - Criar playlist
6 - ---
7 - Ver pontos
8 - Ver conta
9 - Apagar conta
0 - Sair
```

Pesquisar e reproduzir uma música:

```
Opção: 1
Nome da música: Deslocado

*** Deslocado ***
1 - Ouvir música
2 - Adicionar à playlist
0 - Sair
Opção: 1
F para avançar, R para voltar, S para parar a reprodução.
A Tocar: Deslocado
Conto os dias para mim
Com a mala arrumada
...
Fim da lista de reprodução.
Prima S para voltar.
```

Adicionar um álbum à biblioteca

```
Opção: 2
Nome do álbum: Deslocado

*** Deslocado ***
1 - Ouvir álbum
2 - Adicionar à biblioteca
0 - Sair
Opção: 2
Álbum adicionado com sucesso!
```


Criar playlist baseada no histórico

Opção: 5

*** Criar Playlist ***

- 1 - Criar playlist manualmente
- 2 - Criar playlist baseada no histórico
- 3 - Criar playlist explícita baseada no histórico
- 4 - Criar playlist baseada no histórico com duração máxima
- 5 - Criar playlist explícita baseada no histórico com duração máxima
- 0 - Sair

Opção: 2

Playlist adicionada com sucesso!

Ver guardados na biblioteca

** Biblioteca **

- Álbuns:

Álbum: Deslocado

Criado por: NAPA

Músicas:

1. Deslocado

- Playlists:

Playlist: Lista de Favoritos com 6 minutos

Criada por: SpotiUM

Privada

1. Deslocado

2. Ai Coração

Playlist: Lista de Favoritos

Criada por: SpotiUM

Privada

1. Deslocado

2. Ai Coração

3. Grito

Exemplo de exportar o utilizador e álbuns:

```
*** Exportar ***
1 - Exportar utilizadores
2 - Exportar albuns
3 - Exportar playlists
4 - Exportar todos os dados
0 - Sair
Opção: 1
Aviso: O ficheiro já existe e será sobrescrito. Continuar?
1. Para sobrescrever, 2 Para voltar: 1
Dados guardados!

*** Exportar ***
1 - Exportar utilizadores
2 - Exportar albuns
3 - Exportar playlists
4 - Exportar todos os dados
0 - Sair
Opção: 2
Aviso: O ficheiro já existe e será sobrescrito. Continuar?
1. Para sobrescrever, 2 Para voltar: 1
Dados guardados!
```

Exemplo de importar playlists:

```
Opção: 2

*** Importar ***
1 - Importar utilizadores
2 - Importar álbuns
3 - Importar playlists
4 - Importar todos os dados
0 - Sair
Opção: 3
Dados carregados com sucesso!
```

Queries

O programa é capaz de resolver algumas queries simples propostas pelos professores, utilizando informações como por exemplo as músicas guardadas e o histórico de um utilizador. Essas queries são:

→ 1. Qual é a música mais reproduzida?

Esta query determina qual foi a música mais reproduzida por todos os utilizadores. Para isso, percorre o histórico de cada utilizador e contabiliza o número total de vezes que cada música foi ouvida. No final, apresenta a música com maior número de reproduções. É uma métrica direta da popularidade das faixas na plataforma.

```
*** Estatísticas ***
1 - Música mais reproduzida
2 - Intérprete mais escutado
3 - Utilizador que mais músicas ouviu
4 - Utilizador com mais pontos
5 - Género musical mais reproduzido
6 - Número de playlists públicas
7 - Utilizador com mais playlists
0 - Sair
Opção: 1
Música mais reproduzida:
Música: São Bento Grande
Intérprete: Mestre Careca
Editora: CCCB
Género: Capoeira
Duração: 2s
Reproduções: 3
```

→ 2. Qual é o intérprete mais escutado?

Nesta estatística, o sistema identifica o artista (intérprete) que foi mais ouvido entre todas as músicas reproduzidas. Cada vez que uma música é escutada, o nome do seu intérprete é registado e contabilizado. O intérprete com mais ocorrências é apresentado como o mais popular.

```
*** Estatísticas ***
1 - Música mais reproduzida
2 - Intérprete mais escutado
3 - Utilizador que mais músicas ouviu
4 - Utilizador com mais pontos
5 - Género musical mais reproduzido
6 - Número de playlists públicas
7 - Utilizador com mais playlists
0 - Sair
Opção: 2
Intérprete mais escutado: Mestre Careca com 3 reproduções.
```

→ 3. Qual o utilizador que mais músicas ouviu num período ou desde sempre?

Esta query permite identificar o utilizador mais ativo da plataforma, ou seja, aquele que mais músicas ouviu. Esta estatística pode ser consultada de duas formas distintas: sem qualquer filtro temporal, considerando todas as reproduções desde o início da atividade dos utilizadores; ou com um intervalo de datas definido pelo próprio utilizador, sendo então analisadas apenas as reproduções que ocorreram nesse período.

```
*** Estatísticas ***
1 - Música mais reproduzida
2 - Intérprete mais escutado
3 - Utilizador que mais músicas ouviu
4 - Utilizador com mais pontos
5 - Género musical mais reproduzido
6 - Número de playlists públicas
7 - Utilizador com mais playlists
0 - Sair
Opção: 3

*** Filtrar por data? ***
1 - Sim, quero filtrar por data
2 - Não, sem filtro
0 - Sair
Opção: 2
Utilizador que ouviu mais músicas: andreia com 3 músicas ouvidas

*** Filtrar por data? ***
1 - Sim, quero filtrar por data
2 - Não, sem filtro
0 - Sair
Opção: 1
Data inicial (AAAA-MM-DD): 2005-12-13
Data final (AAAA-MM-DD): 2010-03-15
Não existem dados de reprodução.
```

→ 4. Qual é o utilizador com mais pontos?

Esta query apresenta o utilizador com maior pontuação acumulada no sistema. Para isso percorre todos os utilizadores, escrevendo no ecrã o utilizador com mais pontos. A query serve como ranking de desempenho entre utilizadores.

```
*** Estatísticas ***
1 - Música mais reproduzida
2 - Intérprete mais escutado
3 - Utilizador que mais músicas ouviu
4 - Utilizador com mais pontos
5 - Género musical mais reproduzido
6 - Número de playlists públicas
7 - Utilizador com mais playlists
0 - Sair
Opção: 4
Utilizador com mais pontos: Andreia com 133 pontos.
```

→ 5. Qual o tipo de música mais reproduzida?

Esta estatística analisa todas as músicas reproduzidas e identifica qual o género musical mais ouvido. Tal como as anteriores, percorre os históricos de reprodução e conta as ocorrências de cada género. O género com mais repetições é o mais popular.

```
*** Estatísticas ***
1 - Música mais reproduzida
2 - Intérprete mais escutado
3 - Utilizador que mais músicas ouviu
4 - Utilizador com mais pontos
5 - Género musical mais reproduzido
6 - Número de playlists públicas
7 - Utilizador com mais playlists
0 - Sair
Opção: 5
Género musical mais reproduzido: Capoeira com 3 reproduções.
```

→ 6. Quantas playlists públicas existem?

Esta query retorna o número total de playlists públicas existentes na plataforma. Para isso, percorre todas as listas de playlists disponíveis no sistema. Para cada playlist, verifica-se se é pública, incrementando um contador sempre que for o caso.

```
*** Estatísticas ***
1 - Música mais reproduzida
2 - Intérprete mais escutado
3 - Utilizador que mais músicas ouviu
4 - Utilizador com mais pontos
5 - Género musical mais reproduzido
6 - Número de playlists públicas
7 - Utilizador com mais playlists
0 - Sair
Opção: 6
Número de playlists públicas: 1
```

→ 7. Qual o utilizador que tem mais playlists?

Esta query identifica o utilizador que criou mais playlists na plataforma. Para cada utilizador, é consultada a sua biblioteca e contado o número de playlists criadas. Os resultados são armazenados num HashMap<String, Integer>, e é devolvido o utilizador com maior contagem.

```
*** Estatísticas ***
1 - Música mais reproduzida
2 - Intérprete mais escutado
3 - Utilizador que mais músicas ouviu
4 - Utilizador com mais pontos
5 - Género musical mais reproduzido
6 - Número de playlists públicas
7 - Utilizador com mais playlists
0 - Sair
Opção: 7
Utilizador com mais playlists: Andreia com 1 playlists.
```

Conclusão

Este projeto permitiu-nos consolidar os fundamentos da linguagem Java e os conceitos principais da Programação Orientada a Objetos. Ao longo do desenvolvimento, procurámos aplicar boas práticas de design, nomeadamente através da adoção da arquitetura MVC.

Tentamos cumprir todos os objetivos propostos, mas reconhecemos que poderão haver erros ou comportamentos inesperados, nomeadamente em situações limite ou outros casos não testados.

Caso este projeto fosse expandido no futuro, iríamos certificar-nos de uma separação melhor entre as camadas de MVC, assegurando que as partes comunicam como devem; refinar e expandir as exceções personalizadas, como distinguindo entre erros de lógica e validação, de modo a permitir informar melhor o utilizador dos erros; persistência mais robusta dos dados, guardando todos os dados possíveis de forma segura, (como o histórico do utilizador); a criação de testes unitários e automatizados através do JUnit e mais funcionalidades ao programa em si.